

* Dayrun of Codes.

@@. Stack Array Int Demo .java.

```
main {
```

```
    take the capacity in capacity;
```

```
    validation (capacity <= 0);
```

```
    return;
```

passing to constructor.

creating a object of class $\rightarrow$ Stack Array Int() no stack = need $\leftarrow$ (capacity)

```
while (true) {
```

```
    ① push
```

```
    ② pop
```

```
    ③ peek
```

```
    ④ Display
```

```
    ⑤ Exit.
```

choose inputs.

```
switch (choice) {
```

```
    case 1: stack.push (passing value)  $\rightarrow$  method in class.
```

```
    case 2: check condition
```

```
        if (!stack.empty()) {
```

```
            System.out (stack.pop());
```

```
        }
```

```
    case 3: if (stack.isEmpty()) {
```

```
        System.out (stack is empty);
```

```
    } else {
```

```
        System.out (stack.peek());
```

```
    case 4: stack.display();
```

```
    case 5: return;
```

Class,

Stack Array Int Demo {

Declaration of variable. capacity = 4

int [] stack

top = -1;

top = -1 ++top → run only 1

0 → push 5 → 0 → 5 top++
1 → push 6 → 1 → 6 top++
2 → push 7 → 2 → 7 top++
4 → stack.length - 1 4 = 4 return.

StackArrayIntDemo (int capacity) → Constructor initializer.

{
 stack = new int[capacity].
}

Methods: - push (int value) {

 if (top == stack.length - 1) → $4 = 4$ → stack is full.

 {
 System.out.println("Stack is full");
 return;

 } → pre-increment -1 to 0 → zeroth element.

 stack[++top] = value; → value is stored on 0, element.

 System.out.println(value);
}

pop () {

 if (!isEmpty())

 return -1;

 return stack[top--];
}

isEmpty () { check if condition is true
 → return top == -1; means stack is empty.
}

peek () {
 if (isEmpty()) → true
 return -1;

 return stack[top];
}

Top string index.

Display () {

 checking stack is isEmpty if empty return, not print element.

 for (int i = top, i >= 0, i--) {

 System.out.println(stack[i]);

 }

Q2. Reverse string Stack Demo/Java.

main f

takes input of string.

print(reverse(input));

};

Method.

String reverse(String input) {

Stack<Character> stack = new Stack<>();

generic method.
 for (int i = 0; i < input.length(); i++) {
 stack.push(~~stack~~^{input}.charAt(i));
 }
 input character is push at ith index.

StringBuilder result = new StringBuilder();

while (!stack.isEmpty()) {

result.append(stack.pop());

}

return result.toString();

}

Q3. Array basics. Java.

main() {

int marks[] = new int[5];

→ {size of array is 5}

int a[] = {10, 20, 30, 40, 0, 0, 0, 0, 0}; // capacity is 8 4 used.

int bigger[] = Arrays.copyOf(a, a.length * 2);

↓
New array copying the element and doubled the length of array and store into bigger.

// Insert value at index shifting everything off by one step right.

insertAt(int a[], index, size, value) { → Initialize inputs.

(size) < index - 1
a[i] = a[i+1];
shift it;

}

a[index] = value;

{ a[i] is checked index of array and a[i+1] shift index till ~~index~~ condition off next loop if store value at a[index];

Q@. Queue Demo. java ,

int max = 5 → Size of array.

int queue[] = new int[max]; → Defining array.

int front = -1;

int rear = -1;

enqueue() {

if (rear == max-1) → check if queue is full or not.

return;

}

if (front == -1) → Defining front as 0

front = 0;

}

queue[++rear] = value;

}

dequeue() {

if (isEmpty) {

return -1;

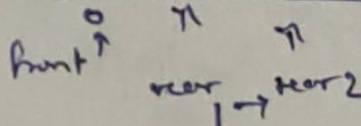
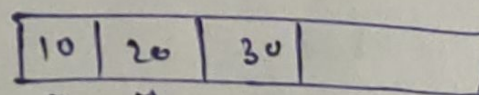
int removed = queue[front++];

if (front > rear) {

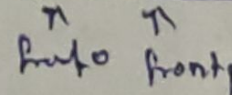
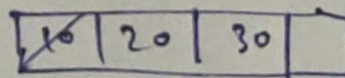
front = -1;

rear = -1;

return removed;



dequeue



0 → 1 And removed from 0 position now front is 1 index..

→ 3 > 2 for the 3 element the condition is true and front = -1; the Queue is empty.
rear = -1;

Q6. Circular Queue Demo. is/are.

At Overall concept like same as the Queue. The only diffing is Queue is in Circular.

is full () { $\rightarrow$ it check the Queue is full or not by size 5 front = 0 and = 4

if (front == 0 && rear == max - 1) || (front == rear + 1);

Enque () {

if (is Empty) {

front = 0;

rear = 0;

else if (front == rear == max - 1) {

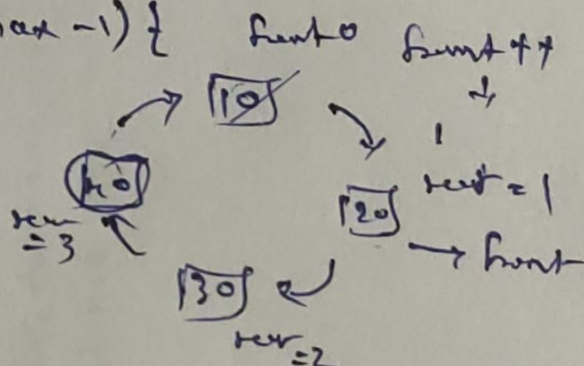
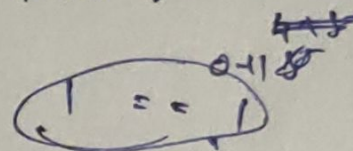
rear = 0

} else { rear ++

}-

queue [rear] = value;

front = 1 == rear + 1



size 4.

Dequeue () {

(is Empty) {

$\rightarrow$ Check the queue is empty if true return. if Not removed element.

}-

int removed = queue [front];

if (front == rear) {

front = -1

rear = -1

else if (front == max - 1) {

front = 0;

else {

front ++;

}-

return removed.

After removing front == rear is same

$\rightarrow$ then make front = -1 that queue is full. rear = -1

front = max size - 1; same 5 = 5;

return front = 0.

else increment front ++

0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3

size 6.